QCon
全球软件开发大会

InfoQ 技术大会

# AI 把代码写完了，谁来验证？

冷乐

TEKO–淘宝App AI自主验证的工程实践



# 泠乐

阿里巴巴 | 高级测试开发专家

## 负责淘宝终端质量

移动端专项测试

APP稳定性和性能体验治理

质量工程与研发效能、
安全生产

## TEKO AI 原生质量体系建设

近年来主导推进AI原生质量体系建设，
聚焦 Agent 工程化、Harness 工程及大模型应用评测。

通过知识、工具和执行环境建设，

用 AI 重构代码评审、单测与自动化验证，
让 AI 能够自主完成质量验证，
切入和改造研发生命周期。

从“工程师逐一验证需求”到“工程师构建验证流水线”，推动AI真正融入研发交付。

# 演讲目录

从可信交付的工程问题，到自主验证的能力体系、真实案例与持续改进


| | | |
|---|---|---|
| 01 | 问题背景 | 编码加速之后，为什么可信交付仍依赖可信验证 |
| 02 | 验证体系 | 判据、可执行环境、信号与环境成熟度 |
| 03 | 实践案例 | 行为、布局、可见性判据如何落成三类测试 |
| 04 | 编排与评估 | teko-qa-skill任务协调，以及对验证的验证 |
| 05 | 落地总结 | 从一条链路起步，明确维护投入与演进方向 |

# 时代浪潮：AI Coding 从辅助工具走向自主 Agent

从补全一行代码，到委派一项任务——研发协作方式正在改变

2024 年以前

## 辅助补全

代码补全 · 行级提示

GitHub Copilot 早期版

2025 年

## 协作开发

多文件编辑 · 对话式开发

Cursor · Claude Code

2026 年

## 任务委派

自主 Agent · 多 Agent 并行

Claude Code · Codex · Antigravity

**传统开发 → AI 辅助开发（人守关卡） → AI-Native（规划、编码、测试、修复）**

**6.42 倍**

淘宝 App 某模块
同期代码变更量

较去年同期增加 数百万行

### Agent Coding Harness：为自主研发建立闭环控制

Gartner 2027 十大战略技术趋势之一：
贯穿软件开发生命周期，管理自主开发者的行为。

**代码生产能力快速增长，可信交付需要同步建设验证与闭环控制能力。**

# 核心矛盾：代码产能爆发，质量验证能力亟待跟上

瓶颈正在从“生成更多代码”，转向“约束和验证机器生成的代码”

## 验证压力：发现 ≠ 消化

**约 1/8**

AI 代码审查
报告有效率

内部 AI 审查系统实测，限本次样本

## PostgreSQL 19：质量门槛抬高

Beta 期间撤回约 8–10 项重要功能。
AI 审查更快暴露边缘问题，
逐项核验与修复仍需要时间。

**启示：提高发现能力，也要建设问题分流、修复与复验能力。**

## 全 AI 闭环，需要补齐四类工程约束

### 01 自证正确性陷阱

生码与自测共享假设，可能共同遗漏业务盲区。

### 02 业务认知边界

语法和测试通过，仍需对齐业务目标与领域规则。

### 03 修复扩散风险

局部修复可能影响依赖，需要重算影响并回归。

### 04 风险判断缺口

影响等级与上线决策，需要明确规则和责任边界。

过去：怎么写得更快？　现在：写完了，谁来验证？怎么验证？

# 从人力验证到AI自主验证，还要跨越3道鸿沟

只让 AI 做需求分析、测试分析与用例生成，仍难以承接持续增长的验证需求

**全人力验证**
人完成全部验证环节

→

**人主导、AI辅助**
AI 生成，人串联与兜底

→

**AI主导、人偶尔介入**
AI 推进，人处理例外

→

**AI自主验证**
围绕目标持续执行与复验

## 01 匹配鸿沟

**用例生成了，却不够贴合业务**

需求、代码与领域知识分散成孤岛；
缺少组织规则、历史经验与变更关联。

**补齐判据融合**
把任务上下文与组织知识串起来

## 02 执行鸿沟

**逻辑正确，却无法真正运行**

前置状态和数据难准备；
缺少稳定入口、触发与观测能力。

**补齐可执行环境与信号**
让场景可触发、结果可判断

## 03 自主鸿沟

**能够执行，却仍需要人串联**

工具能力没有开放给 Agent；
调度、等待、恢复与复跑依赖人工。

**补齐能力接入与闭环控制**
让 Agent 持续推进并按条件结束

**自主验证的关键：把“生成一份用例”推进为“独立完成一条有证据的验证链路”。**

# AI 把代码写完之后，谁来验证?

验证，成为软件工程新的核心命题

## 淘宝 App：面向亿级用户的超级 App

面对 AI Coding 带来的代码产能增长，TEKO 探索 AI 自主验证的工程实践。

### 01 独立验证

如何构建独立于 AI 生成代码
的验证体系，打破“自证闭环”?

**以独立判据与真实证据校验结果**

### 02 业务语义

如何让 AI 理解业务目标，
让验证超越“跑通即可”?

**融合需求、领域知识与组织经验**

### 03 工程落地

如何在超大规模 App 中，
实现 AI 验证的工程化落地?

**贯通环境、信号、工具与闭环控制**

TEKO：为需求交付建立可信判据、可执行验证与可复核证据。

# 从代码生成到自主验证，需要补齐四个条件

AI放大了代码生产速度，但验证能力没有同步跃升；增加测试人力，无法根本解决自主验证的瓶颈。

| 条件 | 核心问题 | TEKO的建设方向 |
| --- | --- | --- |
| **判据** | 什么算对? | 将需求、技术方案、基建规范、团队约束和历史经验，<br>转化为结构化验收条件，并回喂生码 |
| **可执行环境** | 如何让验证真正发生? | 让环境与数据可准备，让App在不同技术分层<br>可寻址、可驱动、可重置、可复现 |
| **信号** | 据什么判断对不对? | 提供屏幕、日志、埋点、运行时状态、接口响应<br>与断言结果等客观证据 |
| **闭环控制** | 下一步做什么，何时结束? | 根据判据和证据决定补充观测、重试、复跑、<br>结束或交给人处理 |

**链路连乘估算：环境60% × 数据50% × 执行40% × 工具自动化70% = 可执行率8.4%[1]**

**验收条件前移**
判据先行、指导生码

**可测性下沉**
全局切面、SDK基线与CI卡口

**验证形成闭环**
执行、观测、判断与复跑

**判据定义预期，环境承载行动，信号提供证据，闭环控制推动验证。**

演进：自主验证 → 代码修复 → 自主研发闭环；线上Crash与舆情 → 回归用例 → 用例库持续生长

# 总体架构：四个条件如何落到工程体系

TEKO [TE-ko] = Testing Expert & Knowledge Orchestrator




## 01 判据融合

融合需求、代码、设计与组织经验，形成共享上下文和结构化验收条件。

## 02 环境可执行

环境与数据可准备，App分层可寻址、可驱动、可重置、可复现。

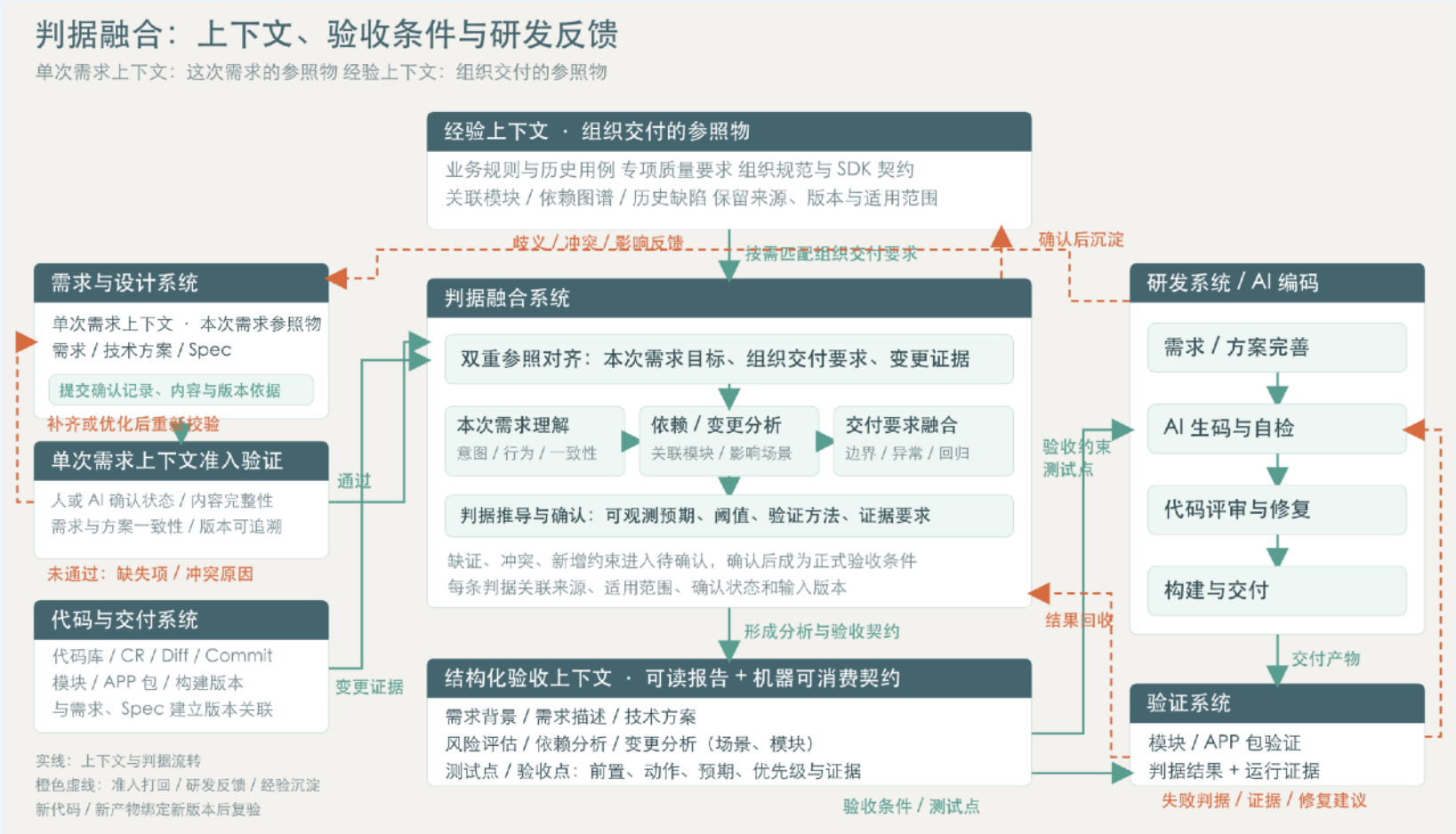
## 03 证据可观测

采集屏幕、日志、埋点、运行时状态与断言结果，以客观证据判断对错。

## 04 验证可闭环

依据判据和证据决定补充观测、重试复跑、结束或转人工，并反馈AI编码持续修正。

# 判据融合：让验收条件参与AI研发全过程




## 需求参照物

单次需求上下文定义本次交付目标；需求与Spec经人或AI确认，保留依据和版本。

## 组织交付参照物

经验上下文承载业务规则、专项质量要求和组织交付标准，明确共同要求。

## 融合判据并回喂

对齐需求目标与适用交付要求，形成验收条件和测试点，约束生码并驱动验证、修复与复验。

# 七类内容形成完整验收上下文

需求理解、影响分析与验收条件相互引用，支持人阅读与研发系统消费

| | |
|---|---|
| 需求背景 | 业务问题、目标、成功标准与范围 |
| 需求描述 | 用户场景、功能行为、边界与明确排除项 |
| 技术方案 | 设计决策、接口契约、状态与异常行为 |
| 风险评估 | 风险原因、触发条件、影响与验证优先级 |
| 依赖分析 | 服务、SDK、配置、数据、环境与版本要求 |
| 变更分析 | 代码改动、关联模块、受影响业务场景与回归范围 |
| 测试点 / 验收点 | 前置条件、动作、预期结果、阈值、验证方法与证据要求 |

共同元数据：来源引用、确认状态、适用范围，以及需求 / Spec / 代码 / 构建版本。

# 判据示例：日历操作串行化

将需求目标与稳定性经验转化为前置条件、操作、预期结果和证据要求

## AC01 并发请求按约定顺序执行

前置：权限已授权，有可核验事件。
动作：连续添加、查询、删除。
预期：API无重叠，顺序及最终状态符合Spec。

## AC02 单次失败结束，后续队列可用

动作：注入一次API失败，再发正常请求。
预期：按错误契约结束，回调次数符合约定，
后续请求继续执行。

## AC03 专项风险有标准、有证据

检查崩溃、积压延迟与持续运行内存变化。
并发规模、时长与阈值待确认。
证据：调用时序、回调、崩溃与性能数据。

## 平台分析：需求上下文与风险




## 局部放大：功能回归与性能风险

1 功能回归风险：addEvent/removeEvent/checkEventExist方法签名由async改为同步void，需确认所有调用方均通过callback接收结果而非await返回值，否则可能导致调用方逻辑异常；需回归验证日历订阅、取消订阅、事件查询全流程功能正确性。

2 性能风险：串行化导致高并发场景下日历操作响应延迟线性增长，若队列积压过多可能影响用户体验；需评估实际业务中并发调用频率，确认串行化不会造成明显卡顿。

判据关联需求与版本，运行证据支撑逐条判断，失败反馈驱动修复与复验。

# 多端、多层协作的电商App，需要怎样的可执行环境?

一条购物链路，贯穿页面、客户端模块、系统与云端服务




搜索 / 推荐 → 商品详情 → 购物车 → 下单支付 → 订单售后

| 执行层次 | 要解决的问题 | 环境能力 |
| --- | --- | --- |
| 代码 / 模块层 | 复杂调用与异常分支，<br>页面难触达 | 直接调用、模拟依赖、<br>低成本构造场景 |
| 宿主 / 集成层 | 生命周期、线程与跨模块协作，<br>局部难验证 | 真实运行时、可控状态、<br>协作验证 |
| 整包 / 真机层 | 端云变化与资源累积，<br>单次成功难说明稳定 | 完整链路、混沌注入、<br>持续压力 |

跨层支撑：Android / iOS / 鸿蒙；关联返回值、日志、资源指标与页面表现

分层执行，让场景可触发、过程可观测、结果可复验。

# 可执行环境：贯通设备、运行时与服务端能力




## 01 分层执行

根据验证目标选择单元、集成、系统或端到端测试，并配齐相应的规范与知识依赖。

## 02 统一底座

移动真机平台、TaoAppUse、三端自动化框架和插件系统，连接设备、App运行时与服务端能力。

## 03 AI友好接入

以CLI、MCP、Skill暴露能力，让Agent理解页面、执行操作，结合日志、截图、ViewTree与状态验证。

# 从可执行环境到分层验证

按风险选择执行入口，形成从代码逻辑到用户体验的验证证据

| 验证手段 | 核心问题 | 验证证据 |
| --- | --- | --- |
| 单元测试 | 局部逻辑和异常分支<br>是否正确? | 返回值、状态断言、<br>分支覆盖率 |
| 运行时白盒 | 真实App内部的模块协作<br>是否可靠? | 内部状态、回调结果、<br>运行日志、崩溃信息 |
| UI测试 | 用户能否正确完成页面操作<br>和业务流程? | 页面断言、截图、业务结果；<br>按需关联服务端校验 |

**局部逻辑 → 真实运行时协作 → 页面交互与业务链路**

单测深入逻辑分支，白盒验证运行时协作，UI测试连接页面交互与业务结果。

# TaoAppUse架构与验证信号




## 四类信号，补齐验证现场

现场、行为、诊断、环境分别回答：看到了什么、执行了什么、内部发生了什么、在什么条件下发生。

## 三类组件，同层协作

Yuda / Civita、Tao App Use SDK与uniapp-device CLI，提供设备操作、端内观测、会话及环境管理。

## 人与Agent，共享证据

Platform和MCP / CLI共用现场与结果，让人工操作衔接Agent分析和验证。

产品规划视图；具体能力以落地进度为准。

# 验证信号：可信的证据，充分的判断

先验证信号是否可信，再判断证据是否足以证明结果符合预期；多种证据相互印证

| 验证维度 | 主要证据 | 验证内容 |
| --- | --- | --- |
| 静态视觉 | 截图 | 布局、文案、图片与控件状态；<br>检查遮挡、错位、截断 |
| 时序与动态 | 图片序列、视频 | 状态变化、流程连贯性；<br>动画、Toast等瞬态反馈 |
| 日志 | 客户端运行 / 异常日志、业务埋点 | 崩溃与异常、关键逻辑与分支；<br>执行顺序与结果 |
| 性能、体验与资源 | 内存、CPU、FPS、磁盘占用，<br>首屏时间、可交互时间等 | 预设阈值、基线退化；<br>异常峰值或持续增长 |
| 内部状态 | TaoAppUse获取的内存状态 | 页面对应的对象、变量及业务状态；<br>状态流转是否符合预期 |
| 外部状态 | 服务端接口返回、数据库字段、<br>服务端日志 | 请求处理、数据落库；<br>客户端与服务端状态一致性 |

### 明确预期

定义正确结果、性能阈值与允许误差

### 证据对齐

同一次操作、设备与账号；
以时间戳或请求标识关联

### 结论有边界

证据缺失或矛盾时标记“待验证”；
以证据覆盖范围界定结论

**“收藏成功”需交叉检查：页面收藏状态、成功反馈、客户端内部状态与服务端收藏记录。**

# 验证环境成熟度：Agent能做到哪一步？

可执行环境 + 验证信号 = 验证环境 | 衡量App、设备、测试工具和执行平台对Agent的支持程度

| 等级 | 如何自评 | 升级要补什么 | 解锁场景 | 主要代价 |
| --- | --- | --- | --- | --- |
| L0 人驱动 | 离开人工，任务无法推进 | 设备、安装包、操作说明 | 人工探索、AI辅助分析 | 重复劳动、依赖经验 |
| L1 能操作 | 看屏幕→操作→再观察 | 截图、输入、安装启动接口 | 简单流程、低频探索 | 速度慢、误点与UI变化 |
| L2 稳定判断 | 重复定位目标，判断操作结果 | UI树、标识、等待、基础断言 | 核心链路、冒烟测试 | 定位维护、技术栈适配 |
| L3 状态可控 | 自动准备条件，失败后恢复 | 数据、登录态、测试入口、重置 | 分支、异常、模块测试 | 业务改造、权限与状态 |
| L4 规模运行 | 并发隔离，中断可接管恢复 | 设备池、独立会话、持久化归档 | 批量回归、多端、CI | 平台、资源与运维 |
| L5 质量闭环 | 失败可归因，修复后可复验 | 判据—用例—缺陷关联与复验 | 持续评估、准出辅助 | 系统集成、误判治理 |

**稳定可用才算升级：成功率 · 人工介入次数 · 恢复能力 · 单次成本；不同模块可处于不同等级。**

---

从目标场景出发，识别当前瓶颈，优先补齐下一阶段所需的验证能力。

# 验证环境建设：设备会话与Agent会话解耦

Agent中断后，保留设备现场与检查点，让任务可以接管或恢复

## Agent会话 | 可替换的执行者

对话与推理上下文、执行进程、工具调用记录
持有设备会话ID，保存任务检查点
中断后，由新的Agent重新连接

## 设备会话 | 独立管理的现场

设备租约、App状态、登录态、连接、日志采集
独立服务负责生命周期与资源管理
依据租约保留现场，支持重新观察和接管

**执行中断 → 保留设备与检查点 → 新Agent重连 → 观察当前状态 → 确认后继续**

### 解锁：长任务可接管与恢复

例如第20分钟中断，有机会从当前状态继续，
减少重新租设备、登录和准备数据。

### 代价：管理恢复过程的不确定性

租约续期、超时回收、并发控制与状态校验。
恢复前先确认上次提交是否成功，避免重复操作。

我们在哪一级？瓶颈是什么？为下一类测试场景，值得补哪些环境能力？

# 从判据到可执行用例：逐层补齐验证条件

逐层明确验证目标、场景与执行条件，再通过真实运行取得证据

**判据：怎样算对 → 测试点：验证什么 → 逻辑用例：怎样验证 → 可执行用例：绑定系统 → 执行取证**

| 阶段 | 这一步补什么 | AI能做什么 | 依赖的可测性能力 |
| --- | --- | --- | --- |
| 判据 → 测试点 | 展开正常、异常、边界与影响场景，建立与判据的对应关系 | 结合需求、代码与经验识别风险，补充场景、检查遗漏 | 系统边界、依赖关系、可观测对象 |
| 测试点 → 逻辑用例 | 补齐前置状态、输入、操作顺序、预期结果与判定方法 | 设计场景组合、步骤与断言，标记歧义及缺失条件 | 状态可构造、故障可触发、结果有证据可判断 |
| 逻辑 → 可执行用例 | 绑定版本、环境、数据与入口，补齐等待、取证和清理机制 | 复用工具生成脚本、绑定参数，试跑并依据失败证据修正 | 数据构造、稳定入口、故障注入、状态观测与环境重置 |

**判据**
失败后队列
仍可继续

**测试点**
异常操作
不阻塞队列

**逻辑用例**
提交A、B、C，使B失败；
检查B错误返回与C完成

**可执行用例**
绑定API、注入点与断言，
补齐等待和数据清理

AI逐层生成与补全用例；可测性建设决定场景能否触发、结果能否观察、执行能否重复。

# 从证据到执行：AI测试用例生成架构

真实需求中的三类判据，如何转为可执行用例




## 证据召回

融合本仓源码、调用图谱、跨仓关系与App图谱 / 业务知识库，关联变更与业务场景。

## 用例生成

模型提炼条件、刺激、观测与预期，分别生成UI回归、白盒设计和单测意图。

## 可执行性校验

脚本完成协议适配与契约校验，模型定向修复，保持原始验证目标。

## 交付与执行

合并有效用例，由下游绑定资源、实施测试，并回收真实结果。

## 职责分工

模型负责理解与设计，脚本保障流程与质量，下游执行器负责实施与验证。

**有证据、有触发、有观测、有断言，才能从“生成用例”走向“有效验证”。**

# 真实需求中的三类判据，如何转为可执行用例

将判据、执行入口与验证信号对应起来，看同一条工程链路如何落地

**判据 → 测试点 → 可执行用例 → 真实执行 → 结果证据**

**案例1 | 单测**
行为判据

**异常容错：命中才兜底，不误吞异常**
真实入口调用 → 返回值与日志断言

**案例2 | 白盒**
布局判据

**多语言适配：尺寸、排列与居中规则**
进程内构造组件 → frame与属性断言

**案例3 | UI自动化**
可见性判据

**同一需求：用户看到的文案完整、无重叠**
触发真实错误页 → 截图与页面检查

案例1来自异常容错需求；案例2与案例3来自同一多语言适配需求，分别检查内部布局与用户可见结果。

# 案例1｜单测：把异常容错判据转成断言

真实需求：组件主类加载失败时允许兜底，其他异常保持原处理路径

| 判据：什么行为才正确 | 测试点：哪些边界要验证 | 单测意图：如何执行验证 |
| --- | --- | --- |
| **命中才兜底**<br>功能启用，进程、事件、异常类型及类名均匹配 | 全条件命中；关闭功能；<br>进程、事件或异常类型不符 | 控制配置与进程，构造事件和异常链；<br>调用真实入口，检查返回值及监控行为 |
| **不误吞异常**<br>缺失类必须是待创建的组件主类 | 缺失类与组件主类不一致 | 创建ComponentA，异常指向HelperB；<br>调用入口，断言返回false |
| **兜底自身可失败**<br>内部异常不能引发二次故障 | 容错逻辑内部抛异常 | 构造内部失败条件，<br>检查是否安全回到原路径 |

**“不能误吞” → 构造两个不同类名 → 调用统一入口 → assertFalse(result)**

类名已脱敏；断言代码为示意。

# 案例1｜单测：从用例描述到可执行断言

以“全条件命中”为例：准备条件、调用真实入口、检查返回值与日志行为

## 用例描述：输入、动作、预期

### 01 构造命中条件

控制进程与配置，构造事件；
异常链中的缺失类与组件主类一致。

### 02 调用真实入口

传入事件与异常，取得处理结果。

### 03 核验结果与副作用

断言返回 true；
验证日志调用及关键参数。

## 单测实现：Arrange → Act → Assert

```
// Arrange：控制依赖，构造输入
setProcessName(TARGET_PROCESS);
setFeatureEnabled(true);
Message msg = createMessage(TARGET_EVENT);
Exception ex = buildExceptionChain("ComponentA");

// Act：调用真实入口
boolean result = handler.handleException(msg, ex);

// Assert：返回值 + 日志行为
assertTrue(result);
verifyLogContains("ComponentA");
```

真实代码节选与脱敏简化；辅助方法、常量及类名采用通用命名

同一验证目标贯穿描述与代码：命中条件可构造，真实入口可调用，结果与副作用可断言。

# 案例1｜单测：7个场景落到Android JUnit

从行为判据出发，生成带明确断言的单测，并完成实际执行

**控制依赖 → 构造输入**
**调用真实入口 → 检查断言**

## 1条正向命中

全条件满足

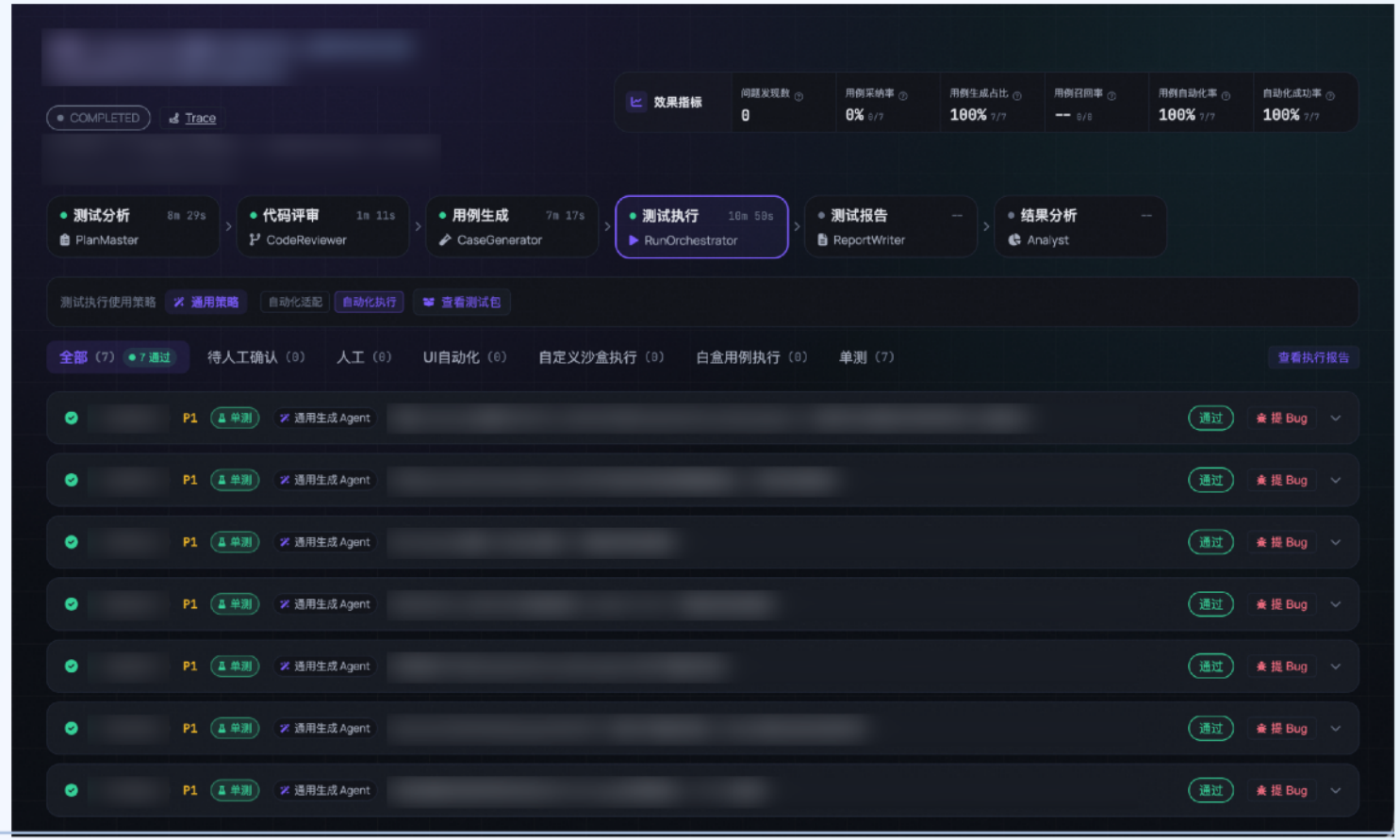
## 5条条件排除

功能、进程、事件、
异常类型与类名不匹配

## 1条异常保护

内部异常安全回到原路径

## Android JUnit：7/7通过




本次用例集7/7通过；覆盖率未记录，真实启动存活还需集成验证。

# 案例2｜白盒：把布局规则转成运行时属性断言

同一多语言适配需求：按钮尺寸自适应，多个按钮横向放不下时自动纵向排列

| 判据：什么行为才正确 | 测试点：验证哪些边界 | 白盒意图：如何执行验证 |
| --- | --- | --- |
| 尺寸约束：短文案保留最小高度；<br>长文案限宽并自适应高度 | 短文案、长文案、<br>换行及高度边界 | App进程内构造组件并触发布局，<br>读取按钮尺寸与标题属性 |
| 排列约束：总宽超出时纵排，<br>否则保持横排 | 多个按钮超宽 / 未超宽 | 构造多个按钮，读取布局后frame，<br>检查横纵坐标关系 |
| 单按钮约束：居中，<br>不触发多按钮纵排逻辑 | 单按钮场景 | 读取排列方式，按按钮自身宽度<br>与容器宽度检查居中位置 |

**“超宽后纵排” → 构造长文案按钮 → 触发布局 → 读取坐标 → 检查 $y_2 > y_1$**

**平台判定标准**

```
判定标准
单按钮横向居中，X坐标接近(375-width)/2
```

**同例返回值**

```
buttonX = 76
expectedCenterX = 127.5
isCentered = false
buttonWidth = 120
```

居中按各按钮自身宽度计算。
将期望坐标与实际坐标对照，
把布局要求落成明确的属性断言。

**白盒把布局规则转成可计算的属性断言，让尺寸、排列与居中从描述走向运行时验证。**

# 案例3｜UI自动化：把可见性判据转成页面校验

同一多语言适配需求的用户视角：按钮文案完整可见，无截断、无重叠

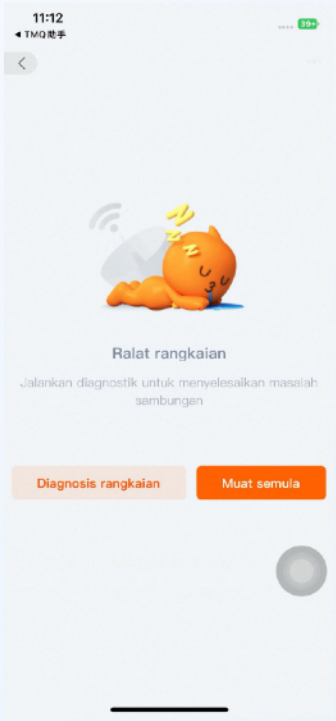
## 验证链路：从判据到页面检查

| | |
|---|---|
| **判据** | 多语言按钮文案完整，无截断、无重叠 |
| **测试点** | 马来语错误页：重试按钮与相邻按钮 |
| **自动化意图** | 切换语言 → 进入目标页 → 断网刷新<br>→ 检查错误页按钮 |
| **页面校验** | 两处文案完整显示；<br>双按钮横向排列，无可见截断或重叠 |

## 马来语场景 · 实际页面证据

**真实错误页**

**两处按钮文案完整**
**横向排列，无重叠**




UI自动化把可见性判据转成操作路径与页面证据，让用户实际看到的结果可以直接核对。

# teko-qa-skill：把分层验证组织成完整任务

统一入口，统一上下文、流程协调和过程观测；阶段能力保持专业分工。

## 收集上下文

汇聚用户意图、需求与技术方案，
关联代码变更、测试包及历史产物。
随任务推进持续补全。

## 串联前后事实

传递上游分析、版本、风险与用例，
让生成、执行与评估
基于一致的事实。

## 统一采集与上报

关联调用、耗时、状态、产物，
记录失败原因与人工介入，
形成可追溯的过程数据。

**任务主线：sessionId → stageTaskId → 版本与阶段产物 → 执行证据与过程数据**

### 入口Agent + Skill

输入解析、意图路由、依赖检查
任务续接、统一采集与交付校验

### Teko后端与阶段能力

保存任务事实与业务策略快照
专业完成分析、生成与真实执行

### 质量评估与交付

规则引擎形成质量结论
Agent汇总证据，校验后交付

入口承担任务级协调与观测；阶段Agent负责专业分析、生成、执行和评估。

# teko-qa-skill：按依赖编排阶段任务

输入齐备即可启动对应轨道；阶段共享任务上下文，并将产物与证据汇入评估




依赖决定先后，输入决定可启动的轨道；普通自动化与白盒证据共同进入评估。

# teko-qa-skill：分阶段交接，沿同一任务续接

需求、代码与测试包可以分次补充；每次续接先查询状态，再核对版本与产物

| 入口模式 | 输入与工作范围 | 交付 |
| --- | --- | --- |
| review | 需求 / 方案 → 分析、初始用例 | READY：分析产物可交接 |
| generate | 代码变更 → 评审、生成 / 召回 / 增强 | READY：用例与执行素材可交接 |
| execute | 同一Session补测试包 → 执行与评估 | 输出质量结论及运行证据 |
| full | 按已提供输入推进完整流程 | 统一检查依赖与证据完整性 |

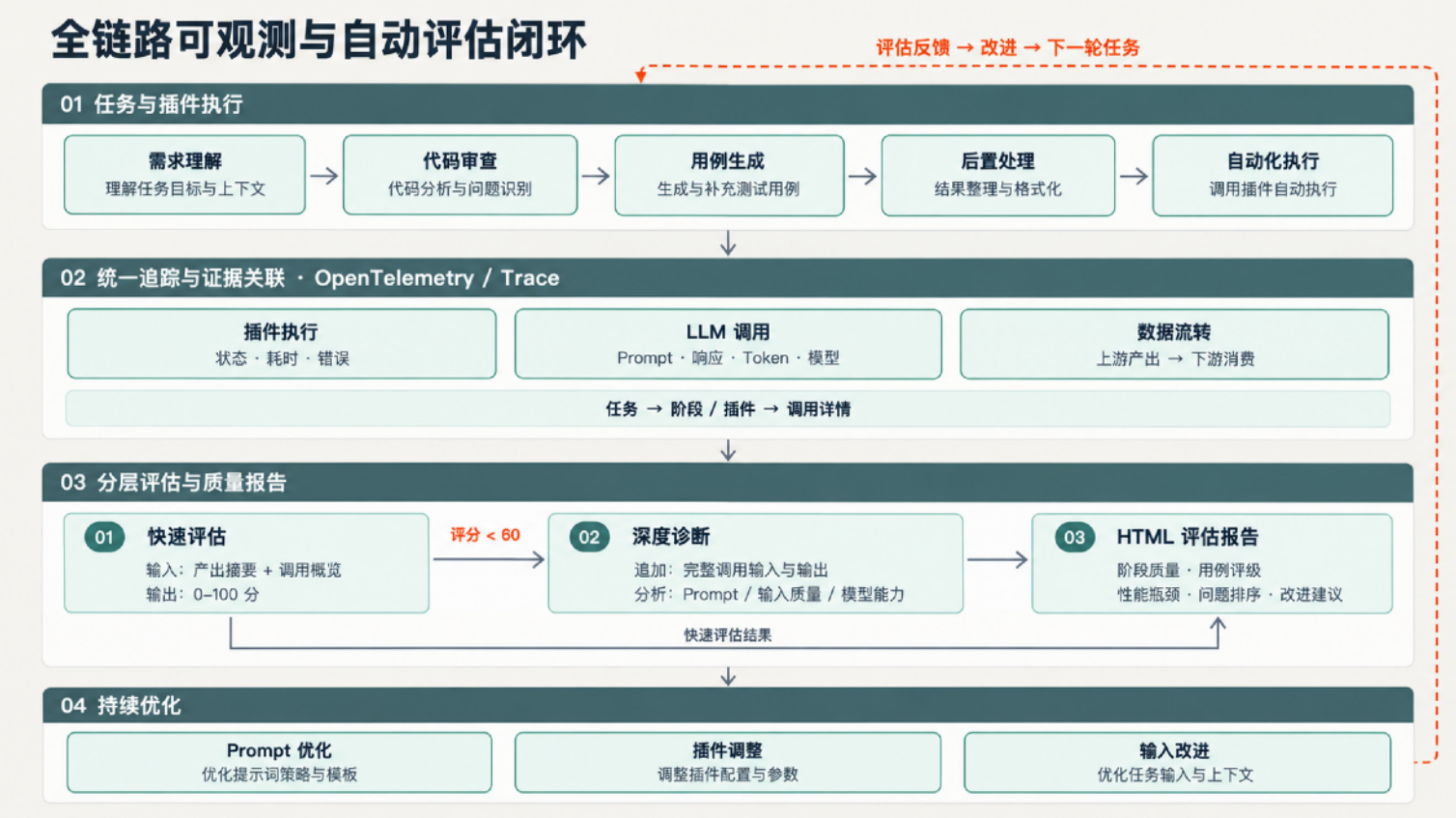
**续接：查询overview → 核对输入与版本 → 复用匹配产物 → 补齐未完成阶段**

**质量状态：PASS / RISK_TO_CONFIRM / FAIL / INSUFFICIENT / BLOCKED**

结果对照原始验收条件；重试与恢复可以调整执行方式，但不能降低验收标准。

# 对验证的验证：全链路可观测与自动评估

记录执行证据 → 分层评估 → 定位问题 → 指导改进




## 统一链路

Trace关联任务、插件与LLM调用，
追踪上下游数据流转。

## 完整证据

状态、耗时、错误、Prompt、
响应、Token与模型标识。

## 分层评估

摘要先评分；低于60分，
使用完整调用详情诊断。

## 反馈优化

报告给出问题与建议，
指导Prompt、插件和输入改进。

让每次验证有可回溯的过程，让产出质量有可检查的依据，让改进有明确方向。

# 真实Trace：从流程完成，继续检查产出有效性

鸿蒙任务实例 | 阶段视图 → 调用详情 → 已记录的输入与输出




## 过程与产出分别检查

页面总耗时：10分13秒

阶段进度：5/6

自动化执行：成功，3分1秒

**根据任务目标，追查空结果来源，**
**核对上游输入与结果归档。**

```
{
  "structuredOutput": {
    "output": { "testCases": [] }, "cases": [],
    "executionStatus": "COMPLETED"
  },
  "executionRoute": "REMOTE_AGENT",
  "testCaseCount": 0,
  "resultSource": "OUTPUT_STRUCTURE"
}
```

执行状态、有效产出与验证覆盖分别评价；Trace为检查“目标是否真正被验证”提供入口。

# 分层评估：筛查、归因与改进回流

先用轻量摘要识别问题，再按需投入完整上下文做深度诊断

### 01 快速筛查

产出摘要 + 调用概览

输出0–100分

控制评估成本与耗时

### 02 按需深诊

评分低于60分

追加完整调用输入输出

分析输入、Prompt与模型适配

### 03 多维报告

阶段质量、用例评级

性能瓶颈、按严重度排序的问题

输出定位线索与改进建议

**用例评级：HIGH 80–100 | MEDIUM 60–79 | LOW 1–59 | UNAVAILABLE 0**

**性能评级：EXCELLENT / GOOD / ACCEPTABLE / SLOW**

**报告建议 → 工程师调整Prompt、插件与输入 → 后续任务 → 再观测、再评估**

模型评分辅助判断，运行证据支撑归因；把评估反馈转成可追踪的工程改进。

# 如何落地：先闭合一条链路，再持续维护与扩展

从高频变更场景起步，把建设路径、维护责任与投入指标一起设计

### 01 选定场景与判据

选一个频繁变化的模块
固定版本、环境与业务范围
明确条件、动作、预期和证据

### 02 跑通可复验链路

准备状态，调用真实执行入口
建立断言，回收原始证据
再让Agent完成适配与协调

### 03 按收益持续扩展

从单场景扩展至业务与多端
持续记录效果和维护投入
按证据优化工具、知识与规则

## 同步明确维护责任：让验证能力成为可持续复用的工程资产

| | |
|---|---|
| **环境与设备** | 维护工具链、测试包、设备状态；记录准备与恢复耗时 |
| **知识与规则** | 维护来源、版本、业务约定；记录过期知识与人工介入 |
| **断言与平台** | 维护信号、断言和采集能力；记录误判、资源与接入成本 |

**先让一个目标稳定可验，再依据成功率、人工介入、恢复能力与单次成本决定扩展。**

# 可信交付，建立在可执行、可观测、可评估的验证之上

四个条件支撑自主验证；对验证本身持续评估，形成可信的工程闭环

| 判据 | 可执行环境 | 信号 | 闭环控制 |
| --- | --- | --- | --- |
| 什么算对 | 如何真正发生 | 据什么判断 | 下一步与何时结束 |
| 需求目标与组织经验<br>形成结构化验收条件 | 准备数据与状态<br>按层次调用真实入口 | 关联页面、日志与状态<br>形成可核对的执行证据 | 协调依赖与任务续接<br>按原始判据评估和复验 |

**对验证的验证：过程可追溯，产出可评估，改进有依据**

真实执行留下证据 → 分层评估定位问题 → 工程改进 → 下一轮持续验证

工程师把经验沉淀为判据、环境、信号与控制机制，让AI持续承担可追溯的质量验证。

# AiCon 北京站

全球人工智能开发与应用大会

## 探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

### 专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

### 专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

### 专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

### 专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

### 专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

### 专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

# THANKS

让每个结论都有证据

判据、可测环境与持续反馈



# 保持交流，一起探索 AI 自主验证



tin

北京 海淀

扫二维码，添加我为朋友。